

CSCI 4100

Operating Systems and Architecture

C Programming

Topics

- The Basics of C
- Functions and Program Structure
- Pointers, Arrays, and Structures
- Input and Output

Introduction to C

- The C programming language was designed for the original UNIX operating system by Dennis Ritchie.
- C and C++ are the most widely used languages for writing operating systems and system utility programs.
- C has high-level features like data types and control structures, as well as low-level features like pointers and explicit memory allocation.
- Most modern programming languages are influenced directly (C++, Objective C, C#, Java) or indirectly (Python, JavaScript, PHP) by C.

Data Types

- There are four basic data types: `char`, `int`, `float`, and `double`.
- Types can be further qualified using `short (int)`, `long (int, double)`, `signed (char, int)`, and `unsigned (char, int)`.
- Boolean values are expressed as `ints` (`0` is false, all other values are true).
- Strings are expressed as arrays of `char` values.
- An integer constant is interpreted as a `signed int` unless otherwise specified (e.g. `45L`, `22U`, `8UL`).
- A floating-point constant is interpreted as a `double` unless otherwise specified (e.g. `3.14F`, `0.02L`).
- Character constants are written in single quotes and may be written using escape sequences like `'\n'` or `'\"'`.
- String constants are written in double quotes and may contain escape sequences (e.g. `"Hello world!\n"`).

Variables

- Variable names can contain letters, digits, or underscores, and must start with a letter or underscore.
- Variables must be declared before they are used, and may be initialized when they are declared.
- Variables with the same type can be declared on the same line.
`int lower, upper, step;
char c, line[1000];`
- The qualifier `const` specifies that the variable may not be changed.
`const double e = 2.71828182845905;
const char msg[] = "warning: ";`

Operators

C operators listed from highest precedence to lowest:

- `() []`
- `! + - ++ --` (unary, right-to-left)
- `* / %`
- `+ -` (binary)
- `> < <= >=`
- `== !=`
- `&&` (short-circuit)
- `||` (short-circuit)
- `?:`
- `= += -= *= /= %=` (right-to-left)

Control Flow

- Declarations and statements can be grouped together into blocks using curly braces.
- C supports `if`, `if-else`, `switch`, `while`, `do-while`, and `for` statements.
- The `break` keyword can be used to break out of a `switch` statement or a loop.
- The `continue` keyword can be used to skip the remainder of the body of a loop in an iteration.
- The `goto` keyword is supported, but should be avoided unless absolutely required.

Functions

- A function definition has a **function header** and a **function body**.
- The function header contains the following:
 - The return type (void if no return value)
 - The name of the function (same rules apply as for variable names)
 - A list of parameters along with their respective types separated by commas and surrounded by parentheses.
- The function body consists of a sequence of statements surrounded by curly braces.
- Example:

```
int square(int num) { return num * num; }
```
- The `main` function is the first function executed in a program.

Function Prototypes

- If functions are used before they are defined, they must first be declared using a **function prototype**.
- A function prototype consists of the header of a function followed by a semicolon.

```
int getline(char line[], int max);  
int strindex(char source[], char searchfor[]);
```
- Variable names are optional in a prototype, but may help clarify what the function does.

Header Files and Preprocessor Statements

- A **header file** contains variable declarations and function prototypes that are defined elsewhere.
- Header file names end in `.h` and regular source file names end in `.c`.
- The `#include` preprocessor directive can be used to include the contents of a header file in a source file before it is compiled.
- Standard library header file names must be surrounded by angle brackets.

```
#include <stdio.h>
```

- User-created header files must be surrounded by double quotes.

```
#include "myfile.h"
```

Hello World Example

```
#include <stdio.h> // includes header file for standard I/O support
```

```
void say_hello(); // function prototype
```

```
// main function
```

```
int main(int argc, char *argv[]) { // function header
```

```
    say_hello(); // function call
```

```
    return 0; // return statement
```

```
}
```

```
void say_hello() { // function header
```

```
    puts("Hello World!"); // output statement
```

```
}
```

Pointers

- A **pointer variable** or **pointer** contains a virtual memory address.
- A pointer can be declared as follows:
`int *ptr; // ptr is a pointer to an int`
- The address of a variable can be accessed using the `&` (address-of) operator.
`ptr = &var; // ptr gets the address of var`
- The value stored at an address can be accessed using the `*` (dereference) operator
`var = *ptr; // var gets the value stored at address ptr`
- The `&` and `*` operators have the same precedence as other unary operators like `+`, `-`, `!`, `++`, and `--`.

Arrays

- An array variable is a pointer to a sequence of values stored contiguously in memory.
- An array can be declared as follows:
`int arr[10]; // arr is a pointer to 10 int values`
- An array can be indexed using the `[]` (array-indexing) operator.
`arr[3] = 42;`
`var = arr[i];`
- An array variable is a constant pointer and can not be modified.
- An indexed array variable can be used anywhere a variable of the base type of the array can be used.
- If an array variable is passed to a function, the function will have access to the array itself, not a copy.

Strings

- In C, a string is represented as a null-terminated array of characters.
- A string literal is a pointer to an array of constant characters.
`const char * message = "Hello World!";`
- The variable can be assigned another value, but the characters in the string can not be modified.
`message = "Goodbye!"; // legal`
`message[7] = '?'; // not legal`
- A character array is a constant pointer to an array of characters.
`char message_copy[9];`
- The variable can not be modified, but the characters in the string can be assigned different values.
`message_copy = message; // not legal`
`message_copy[0] = message[0]; // legal`

Structures

- A **structure** contains one or more variables of possibly different types.
- A structure type can be declared as follows:

```
struct point {  
    double x;  
    double y;  
};
```
- A variable of a structure type can be declared as follows:

```
struct point pt;
```
- Members of a structure can be accessed using the `.` (structure member) operator.

```
pt.x = 4; pt.y = 5;  
dist = sqrt(pt.x * pt.x + pt.y * pt.y);
```

Pointers and Structures

- A pointer to a structure can be created as follows:

```
struct point pt;  
struct point *pt_ref = &pt;
```

- To access a field of a structure via a pointer, you must dereference the pointer, then select the field:

```
(*pt_ref).x = 4.5; // . has higher precedence than *
```

- Alternatively, you can use the -> operator to do both operations:

```
pt_ref->x = 4.5;
```

- A pointer to a field in a structure can be created as follows:

```
int *x_ref = &pt.x; // . has higher precedence than &  
int *y_ref = &pt_ref->y; // -> has higher precedence than &
```


Console Input and Output

- The following functions read from or write to the console one character at a time:

```
int getchar()    // returns the character read
```

```
int putchar(int) // takes the character to write
```

```
// returns the character written
```

- The following functions read from or write to the console one line at a time:

```
char *gets(char *line) // reads a line from the console to a string
```

```
int puts(char *line)    // writes a line from a string to the console
```

- The following functions use a format string for input and output

```
int scanf(char *format, ...); // reads into one or more variables
```

```
// using a format string
```

```
int printf(char *format, ...); // writes from one or more variables
```

```
// using a format string
```

File Input and Output

- The type `FILE *` is a file pointer that points to a structure representing a file.
- Console operations use the built-in `stdin`, `stdout`, and `stderr` file pointers.
- Functions to open and close a file:
`FILE *fopen(char *name, char *mode) // open a file (mode is "r", "w", or "rw")`
`int fclose(FILE *fp) // close a file`
- Functions to read a character from or write a character to a file:
`int fgetc(FILE *fp) // read a character from a file`
`int fputc(int c, FILE *fp) // write a character to a file`
- Functions to read a line from or write a line to a file:
`char *fgets(char *line, int maxline, FILE *fp) // read a line from a file`
`int fputs(char *line, FILE *fp) // write a line to a file`
- Functions to read a line from or write a line to a file:
`int fscanf(FILE *fp, char *format, ...) // read values from a file to variables`
`int fprintf(FILE *fp, char *format, ...) // write a formatted string to a file`